

Guía Docente: Programación de Bases de Datos (RA5)

Profesor de FP - Especialidad Informática

April 17, 2026

Resumen Curricular (DAM/DAW)

Esta guía cubre el **RA5**: "Desarrolla procedimientos almacenados evaluando y utilizando las sentencias del lenguaje incorporado en el sistema gestor de bases de datos".

1 Introducción: ¿Por qué programar en la BBDD?

Normalmente usamos SQL de forma **declarativa** (pedimos qué queremos, no cómo obtenerlo). La programación nos permite usar un estilo **imperativo** para:

- Automatizar tareas repetitivas (CE.a).
- Centralizar reglas de negocio (blindaje).
- Reducir el tráfico de red enviando solo la llamada al procedimiento.

1.1 El Delimitador (DELIMITER)

CE evaluado: CE.b, CE.c

MySQL usa el ; para terminar cada orden. Al escribir un bloque de código (que contiene muchos ;), el cliente de MySQL se confundiría. Por eso, "cambiamos" el fin de orden temporalmente.

Listing 1: Uso de DELIMITER

```
DELIMITER //
```

-- Todo lo que escribas aqui no se ejecutara hasta que pongas //

```
DELIMITER ;
```

2 Procedimientos Almacenados (Stored Procedures)

CE evaluado: CE.f

Un procedimiento es un conjunto de sentencias SQL que se guardan en el servidor. No devuelven un valor como tal (usando RETURN), sino que pueden realizar acciones o devolver parámetros de salida (OUT).

2.1 Ejemplos de Complejidad Creciente

1. Sin parámetros (El "Hola Mundo"):

Listing 2: src/proc_saludo.sql

```
DELIMITER //
CREATE PROCEDURE saludo()
BEGIN
    SELECT '¡Hola clase de DAM!' AS mensaje;
END //
DELIMITER ;

CALL saludo();
```

2. Con parámetro de entrada (IN):

Listing 3: src/proc_buscar_actor.sql

```
DELIMITER //
CREATE PROCEDURE buscar_actor(IN p_apellido VARCHAR(45))
BEGIN
    SELECT first_name, last_name FROM actor
    WHERE last_name LIKE CONCAT(p_apellido, '%');
END //
DELIMITER ;

CALL buscar_actor('Jackman');
```

3. Con parámetro de salida (OUT):

Listing 4: src/proc_contar_peliculas.sql

```
DELIMITER //
CREATE PROCEDURE contar_peliculas(OUT p_total INT)
BEGIN
    SELECT COUNT(*) INTO p_total FROM film;
END //
DELIMITER ;

CALL contar_peliculas(@resultado);
SELECT @resultado;
```

4. Ejemplo Real Sakila (IN y OUT): Este procedimiento permite obtener el total de películas en las que ha participado un actor y su duración media, devolviendo ambos valores mediante parámetros de salida.

Listing 5: src/proc_get_actor_stats.sql

```
DELIMITER //
```

```
CREATE PROCEDURE get_actor_stats(IN p_actor_id INT, OUT p_films INT, OUT p_avg_len
    DECIMAL(10,2))
BEGIN
    SELECT COUNT(*), AVG(length) INTO p_films, p_avg_len
    FROM film JOIN film_actor USING(film_id) WHERE actor_id = p_actor_id;
END //
DELIMITER ;
```

3 Funciones de Usuario (Stored Functions)

CE evaluado: CE.f, CE.e

A diferencia de los procedimientos, las funciones **deben devolver un único valor** (RETURNS) y se pueden usar directamente dentro de un SELECT o un WHERE.

3.1 Características de la Función (Seguridad y Optimización)

MySQL requiere que indiquemos qué tipo de operaciones realizará la función para optimizar su ejecución y garantizar la integridad en entornos con réplicas.

DETERMINISTIC: La función siempre devuelve el mismo resultado para los mismos parámetros de entrada (ej. una operación matemática o un formateo de texto).

NOT DETERMINISTIC: (Valor por defecto) El resultado puede variar aunque los parámetros sean iguales. Ocurre cuando la función usa:

- Funciones de tiempo: NOW(), CURDATE(), SYSDATE().
- Funciones de azar: RAND().
- Datos de tablas: Si la tabla cambia, el resultado de la función cambiará para el mismo ID.

READS SQL DATA: La función contiene sentencias SELECT para leer datos de las tablas, pero no las modifica. Es la más común en Sakila.

MODIFIES SQL DATA: Indica que la función realizará cambios (INSERT, UPDATE, DELETE).

Nota para el Profesor

MySQL es muy estricto con esto por la **replicación**. Si una función no es determinista y se ejecuta en un servidor "esclavo", podría devolver un valor distinto al del "maestro" (ej. por usar NOW()), rompiendo la sincronización de la base de datos.

3.1.1 ¿Cuándo nos interesa usar funciones NO deterministas?

Aunque parezcan "peligrosas" para el SGBD, son esenciales en el día a día:

- **Cálculos dinámicos:** Una función que calcula los días de retraso de un alquiler (DATEDIFF(NOW(), rental_date)) DEBE ser no determinista, porque el resultado cambia cada día que pasa.
- **Seguridad y Azar:** Generar un PIN temporal para un cliente o un token de sesión usando RAND().
- **Estado del Sistema:** Una función que verifique si el servidor está sobrecargado o si una tienda está abierta en el momento de la llamada.

3.1.2 ¿Es obligatorio? Valores por defecto y el Error 1418

Si no escribes estas etiquetas, MySQL aplica por defecto:

- **NOT DETERMINISTIC**
- **CONTAINS SQL** (asume que usas SQL pero no especifica si lees o escribes).

¿Por qué da error entonces? En servidores con la replicación activada (o log binario), MySQL prohíbe crear funciones si no declaras explícitamente que son **DETERMINISTIC**, **NO SQL** o **READS SQL DATA**. El error típico es:

Error Típico

```
ERROR 1418 (HY000): This function has none of DETERMINISTIC, NO SQL,
or READS SQL DATA in its declaration...
```

Solución para el alumno:

1. Ser un buen profesional y declarar siempre la característica (recomendado).
2. Desactivar la restricción (solo para entornos de desarrollo) con:
`SET GLOBAL log_bin_trust_function_creators = 1;`

3.2 Ejemplos de Complejidad Creciente

1. Operación Matemática Simple:

Listing 6: src/func_calcular_iva.sql

```
DELIMITER //
CREATE FUNCTION calcular_iva(p_precio DECIMAL(10,2))
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
    RETURN p_precio * 1.21;
END //
DELIMITER ;
```

2. Formateo de Texto:

Listing 7: src/func_nombre_completo.sql

```
DELIMITER //
CREATE FUNCTION nombre_completo(p_nombre VARCHAR(45), p_apellido VARCHAR(45))
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    RETURN CONCAT(p_apellido, ', ', p_nombre);
END //
DELIMITER ;
```

3. Consulta Simple de BBDD:

Listing 8: src/func_dias_alquiler.sql

```
DELIMITER //
CREATE FUNCTION dias_alquiler(p_rental_id INT)
RETURNS INT READS SQL DATA
BEGIN
```

```

DECLARE v_dias INT;
SELECT DATEDIFF(return_date, rental_date) INTO v_dias
FROM rental WHERE rental_id = p_rental_id;
RETURN v_dias;
END //
DELIMITER ;

```

4. Ejemplo Real Sakila (Lógica Compleja): Esta función es fundamental en Sakila. Determina si una copia concreta (`inventory_id`) está disponible.

Listing 9: `src/func_inventory_in_stock.sql`

```

DELIMITER //
CREATE FUNCTION inventory_in_stock(p_inventory_id INT)
RETURNS BOOLEAN READS SQL DATA
BEGIN
    DECLARE v_rentals INT;
    DECLARE v_out INT;
    SELECT COUNT(*) INTO v_rentals FROM rental WHERE inventory_id = p_inventory_id
    ;
    IF v_rentals = 0 THEN RETURN TRUE; END IF;
    SELECT COUNT(rental_id) INTO v_out
    FROM inventory LEFT JOIN rental USING(inventory_id)
    WHERE inventory.inventory_id = p_inventory_id AND rental.return_date IS NULL;
    IF v_out > 0 THEN RETURN FALSE; ELSE RETURN TRUE; END IF;
END //
DELIMITER ;

```

4 Estructuras de Control (CE.g)

Permiten que nuestro código tome decisiones o repita acciones.

4.1 Ejemplos de Complejidad Creciente

1. Condicional IF simple:

Listing 10: src/ctrl_if.sql

```
IF p_edad >= 18 THEN
    SET p_mensaje = 'Mayor de edad';
ELSE
    SET p_mensaje = 'Menor de edad';
END IF;
```

2. Estructura CASE (Múltiples opciones):

Listing 11: src/ctrl_case.sql

```
CASE p_categoria_id
    WHEN 1 THEN SET p_nombre_cat = 'Acción';
    WHEN 2 THEN SET p_nombre_cat = 'Animación';
    ELSE SET p_nombre_cat = 'Otros';
END CASE;
```

3. Bucle WHILE (Repetición controlada):

Listing 12: src/ctrl_while.sql

```
DECLARE i INT DEFAULT 1;
WHILE i <= 5 DO
    INSERT INTO tabla_log(msg) VALUES (CONCAT('Paso ', i));
    SET i = i + 1;
END WHILE;
```

4. Bucle LOOP con LEAVE (Salida forzada):

Listing 13: src/ctrl_loop.sql

```
mi_bucle: LOOP
    IF v_error = TRUE THEN
        LEAVE mi_bucle;
    END IF;
    -- Realizar acciones...
END LOOP mi_bucle;
```

5. Caso Real: Generación Masiva (Logística Global): Este ejemplo (extraído de la práctica de saneamiento de la UT5) muestra cómo usar un bucle WHILE para insertar 100.000 registros con datos aleatorios y control de progreso.

Listing 14: src/ctrl_masivo.sql

```
DELIMITER //
CREATE PROCEDURE GenerarEnviosMasivos()
BEGIN
    DECLARE i INT DEFAULT 0;
    WHILE i < 100000 DO
```

```

INSERT INTO envios (tracking_number, cliente_id, f_salida, importe_envio)
VALUES (CONCAT('TRK-', FLOOR(RAND()*999999999)),
        FLOOR(1 + RAND() * 500),
        DATE_FORMAT(DATE_ADD('2025-01-01', INTERVAL i MINUTE),
                     ELT(1 + FLOOR(RAND() * 4), '%d/%m/%Y', '%Y-%m-%d', '%d-%m-%y', '%y/%m/%d')),
        CONCAT(ROUND(RAND()*500, 2), ' EUR'));
SET i = i + 1;
IF i % 20000 = 0 THEN
    SELECT CONCAT('... ', i, ' envios procesados.') AS Progreso;
END IF;
END WHILE;
END //
DELIMITER ;

```


5 Disparadores (Triggers)

CE evaluado: CE.h

Un disparador es un bloque de código que MySQL ejecuta **automáticamente** cuando ocurre un evento (INSERT, UPDATE o DELETE) en una tabla específica.

5.1 Conceptos Clave: El "Cuándo" y el "Qué"

1. Momento de Ejecución (BEFORE / AFTER):

- **BEFORE:** Se ejecuta **antes** de que el cambio se guarde en el disco. Ideal para validar datos o modificarlos sobre la marcha. Si lanzas un error aquí (SIGNAL), el cambio no se guarda.
- **AFTER:** Se ejecuta **después** de que el cambio se haya guardado con éxito. Se usa para "efectos secundarios" en otras tablas.

2. Acceso a los Datos (NEW / OLD): Dentro del trigger, podemos acceder a los valores de la fila afectada:

- **NEW:** Los valores que se van a insertar o los nuevos valores tras un UPDATE.
- **OLD:** Los valores que ya existían antes de un UPDATE o los que se van a borrar en un DELETE.

Operación	INSERT	UPDATE	DELETE
NEW	✓ (Disponible)	✓ (Disponible)	—
OLD	—	✓ (Disponible)	✓ (Disponible)

5.2 ¿Para qué se usan en la vida real?

Los disparadores son el "último muro de defensa" de la integridad de los datos:

- **Auditoría Inalterable:** Guardar quién, cuándo y qué cambió en una tabla de pagos sin que la aplicación lo sepa.
- **Sincronización de Totales:** Si insertas una venta, un trigger puede actualizar automáticamente el campo `stock_actual` de la tabla productos.
- **Validaciones de Negocio Complejas:** Impedir que un cliente alquile más de 5 películas si tiene multas pendientes.
- **Caché de BBDD:** Mantener una tabla de "Estadísticas Rápidas" siempre actualizada.

5.3 Ejemplos de Complejidad Creciente

1. Validar/Formatear antes de insertar (BEFORE):

Listing 15: src/trig_capitalizar.sql

```
DELIMITER //
CREATE TRIGGER capitalizar_apellido BEFORE INSERT ON actor
FOR EACH ROW
BEGIN
    SET NEW.last_name = UPPER(NEW.last_name);
END //
DELIMITER ;
```

2. Registro de auditoría simple (AFTER):

Listing 16: src/trig_log_cliente.sql

```
DELIMITER //
CREATE TRIGGER log_nuevo_cliente AFTER INSERT ON customer
FOR EACH ROW
BEGIN
    INSERT INTO logs(accion, id_cliente) VALUES ('NUEVO', NEW.customer_id);
END //
DELIMITER ;
```

3. Impedir una acción (SIGNAL):

Listing 17: src/trig_signal.sql

```
DELIMITER //
CREATE TRIGGER no_borrar_actores BEFORE DELETE ON actor
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Prohibido borrar actores';
END //
DELIMITER ;
```

4. Ejemplo Real Sakila (Sincronización):

Listing 18: src/trig_ins_film.sql

```
DELIMITER //
CREATE TRIGGER ins_film AFTER INSERT ON film
FOR EACH ROW
BEGIN
    INSERT INTO film_text (film_id, title, description)
    VALUES (NEW.film_id, NEW.title, NEW.description);
END //
DELIMITER ;
```

6 Ejercicios para Profundizar

En esta sección se plantean retos que combinan varios de los conceptos aprendidos para resolver problemas de gestión reales sobre Sakila.

6.1 Bloque 1: Procedimientos Almacenados

1. **Alquiler Rápido:** Crea un procedimiento que reciba `customer_id`, `inventory_id` y `staff_id`, e inserte un nuevo registro en `rental` con la fecha actual.
2. **Cierre de Tienda:** Procedimiento que reciba un `store_id` y mueva todos sus empleados a la otra tienda existente.
3. **Actualización de Precios:** Procedimiento que incremente un % (parámetro) el `rental_rate` de todas las películas de una categoría específica.
4. **Estadísticas de Actor:** Procedimiento que devuelva mediante parámetros OUT el número de películas y la duración media de un actor por su ID.
5. **Limpieza de Logs:** Procedimiento que borre registros de una tabla de auditoría (hipotética) más antiguos de N días.

6.2 Bloque 2: Funciones de Usuario

1. **Cálculo de Multas:** Función que reciba un `rental_id` y devuelva la multa (1.50€ por cada día de retraso sobre el `rental_duration`).
2. **Estado de Cliente:** Función que devuelva 'VIP' si el cliente ha gastado más de 150€, 'ESTÁNDAR' en caso contrario.
3. **Conversor de Tiempo:** Función que reciba minutos (INT) y devuelva un texto con formato "Xh Ym".
4. **Disponibilidad Real:** Función que reciba un `film_id` y devuelva el total de copias disponibles (en stock) sumando todas las tiendas.
5. **Email de Marketing:** Función que genere un email corporativo: `nombre.apellido@sakilavideo.c` en minúsculas.

6.3 Bloque 3: Estructuras de Control

1. **Clasificador por Rating:** Script que asigne una descripción larga ('Para todos', 'Mayores de 13', etc.) basándose en el campo `rating` (G, PG, PG-13, R, NC-17).
2. **Generador de Números:** Procedimiento con un bucle que inserte 100 filas de prueba en una tabla auxiliar.
3. **Validación de Stock:** Lógica IF/ELSE que compruebe si una película existe antes de intentar alquilarla.
4. **Promoción Mensual:** CASE que aplique diferentes descuentos según el mes de la fecha actual.
5. **Bucle de Intereses:** Bucle WHILE que calcule el interés compuesto de una deuda ficticia hasta superar un umbral.

6.4 Bloque 4: Disparadores (Triggers)

1. **Auditoría de Pagos:** Trigger que guarde en una tabla `audit_payments` cualquier cambio en el campo `amount`.
2. **Protección de Actores:** Trigger que impida borrar actores que han participado en más de 20 películas.
3. **Auto-Mayúsculas:** Trigger que asegure que el `first_name` de un cliente siempre se guarde en mayúsculas tras un `UPDATE`.
4. **Control de Stock Mínimo:** Trigger que impida alquilar una película si no hay copias disponibles.
5. **Histórico de Email:** Trigger que guarde el email anterior de un cliente antes de que sea modificado.

6.5 Bloque 5: Cursores y Handlers

1. **Resumen de Ventas:** Cursor que recorra todos los empleados y calcule su total de ventas procesadas (`payment`).
2. **Normalizador de Nombres:** Cursor que recorra todos los actores y ponga en mayúscula la primera letra y en minúscula el resto.
3. **Informe por Categoría:** Cursor que genere un reporte de ingresos totales por cada categoría de película.
4. **Aviso de Devolución:** Cursor que identifique alquileres sin devolver y los inserte en una tabla de "Pendientes".
5. **Reemplazo Masivo:** Cursor que busque una palabra en las descripciones de las películas y la reemplace por otra.

7 Solucionario de Profundización (25 Ejercicios)

7.1 Bloque 1: Procedimientos Almacenados

1. Alquiler Rápido:

Listing 19: src/sol_b1_q1.sql

```
DELIMITER //
CREATE PROCEDURE rent_movie(IN p_cust_id INT, IN p_inv_id INT, IN p_staff_id INT)
BEGIN
    INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
    VALUES (NOW(), p_inv_id, p_cust_id, p_staff_id);
END //
DELIMITER ;
```

2. Cierre de Tienda:

Listing 20: src/sol_b1_q2.sql

```
DELIMITER //
CREATE PROCEDURE close_store(IN p_store_id INT)
BEGIN
    DECLARE v_target_store INT;
    SELECT store_id INTO v_target_store FROM store WHERE store_id <> p_store_id
    LIMIT 1;
    UPDATE staff SET store_id = v_target_store WHERE store_id = p_store_id;
END //
DELIMITER ;
```

3. Actualización de Precios:

Listing 21: src/sol_b1_q3.sql

```
DELIMITER //
CREATE PROCEDURE update_cat_prices(IN p_perc DECIMAL(5,2), IN p_cat_id INT)
BEGIN
    UPDATE film f JOIN film_category fc USING(film_id)
    SET f.rental_rate = f.rental_rate * (1 + p_perc/100)
    WHERE fc.category_id = p_cat_id;
END //
DELIMITER ;
```

4. Estadísticas de Actor:

Listing 22: src/sol_b1_q4.sql

```
DELIMITER //
CREATE PROCEDURE get_actor_stats(IN p_actor_id INT, OUT p_films INT, OUT p_avg_len
    DECIMAL(10,2))
BEGIN
    SELECT COUNT(*), AVG(length) INTO p_films, p_avg_len
    FROM film JOIN film_actor USING(film_id) WHERE actor_id = p_actor_id;
END //
DELIMITER ;
```

5. Limpieza de Logs:

Listing 23: src/sol_b1_q5.sql

```
DELIMITER //
CREATE PROCEDURE purge_audit_logs(IN p_days INT)
BEGIN
    DELETE FROM audit_log WHERE log_date < DATE_SUB(NOW(), INTERVAL p_days DAY);
END //
DELIMITER ;
```

7.2 Bloque 2: Funciones de Usuario

1. Cálculo de Multas:

Listing 24: src/sol_b2_q1.sql

```
DELIMITER //
CREATE FUNCTION get_late_fine(p_rental_id INT) RETURNS DECIMAL(10,2) READS SQL
DATA
BEGIN
    DECLARE v_delay INT;
    SELECT DATEDIFF(IFNULL(return_date, NOW()), rental_date) - f.rental_duration
        INTO v_delay
    FROM rental r JOIN inventory i USING(inventory_id) JOIN film f USING(film_id)
    WHERE r.rental_id = p_rental_id;
    RETURN IF(v_delay > 0, v_delay * 1.50, 0.00);
END //
DELIMITER ;
```

2. Estado de Cliente:

Listing 25: src/sol_b2_q2.sql

```
DELIMITER //
CREATE FUNCTION get_customer_status(p_cust_id INT) RETURNS VARCHAR(20) READS SQL
DATA
BEGIN
    DECLARE v_spent DECIMAL(10,2);
    SELECT SUM(amount) INTO v_spent FROM payment WHERE customer_id = p_cust_id;
    RETURN IF(v_spent > 150, 'VIP', 'ESTANDAR');
END //
DELIMITER ;
```

3. Conversor de Tiempo:

Listing 26: src/sol_b2_q3.sql

```
DELIMITER //
CREATE FUNCTION format_min(p_min INT) RETURNS VARCHAR(20) DETERMINISTIC
BEGIN
    RETURN CONCAT(p_min DIV 60, 'h ', p_min MOD 60, 'm');
END //
DELIMITER ;
```

4. Disponibilidad Real:

Listing 27: src/sol_b2_q4.sql

```

DELIMITER //
CREATE FUNCTION total_film_stock(p_film_id INT) RETURNS INT READS SQL DATA
BEGIN
    DECLARE v_count INT;
    SELECT COUNT(*) INTO v_count FROM inventory i
    WHERE film_id = p_film_id AND inventory_in_stock(i.inventory_id);
    RETURN v_count;
END //
DELIMITER ;

```

5. Email de Marketing:

Listing 28: src/sol_b2_q5.sql

```

DELIMITER //
CREATE FUNCTION gen_mkt_email(p_first VARCHAR(45), p_last VARCHAR(45)) RETURNS
    VARCHAR(100) DETERMINISTIC
BEGIN
    RETURN LOWER(CONCAT(p_first, '.', p_last, '@sakilavideo.com'));
END //
DELIMITER ;

```

7.3 Bloque 3: Estructuras de Control

1. Clasificador por Rating:

Listing 29: src/sol_b3_q1.sql

```
DELIMITER //
CREATE PROCEDURE desc_rating(IN p_rating VARCHAR(10), OUT p_desc VARCHAR(100))
BEGIN
    CASE p_rating
        WHEN 'G' THEN SET p_desc = 'Todos los publicos';
        WHEN 'PG' THEN SET p_desc = 'Supervision de padres';
        WHEN 'R' THEN SET p_desc = 'Restringido (+18)';
        ELSE SET p_desc = 'Otras categorias';
    END CASE;
END //
DELIMITER ;
```

2. Generador de Números:

Listing 30: src/sol_b3_q2.sql

```
DELIMITER //
CREATE PROCEDURE populate_test()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 100 DO
        INSERT INTO test_table(val) VALUES (i);
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;
```

3. Validación de Stock:

Listing 31: src/sol_b3_q3.sql

```
DELIMITER //
CREATE PROCEDURE check_and_rent(IN p_film_id INT, IN p_store_id INT)
BEGIN
    DECLARE v_stock INT;
    CALL film_in_stock(p_film_id, p_store_id, v_stock);
    IF v_stock > 0 THEN SELECT 'Disponible' AS msg;
    ELSE SELECT 'Sin existencias' AS msg;
    END IF;
END //
DELIMITER ;
```

4. Promoción Mensual:

Listing 32: src/sol_b3_q4.sql

```
DELIMITER //
CREATE FUNCTION get_monthly_promo() RETURNS DECIMAL(5,2) DETERMINISTIC
BEGIN
    IF MONTH(NOW()) = 12 THEN RETURN 20.00;
    ELSEIF MONTH(NOW()) = 8 THEN RETURN 10.00;
    ELSE RETURN 0.00;
END
```



```

    END IF;
END //
DELIMITER ;

```

5. Bucle de Intereses:

Listing 33: src/sol_b3_q5.sql

```

DELIMITER //
CREATE PROCEDURE calc_debt(IN p_init DECIMAL(10,2), IN p_target DECIMAL(10,2))
BEGIN
    DECLARE v_debt DECIMAL(10,2) DEFAULT p_init;
    WHILE v_debt < p_target DO
        SET v_debt = v_debt * 1.05;
    END WHILE;
    SELECT v_debt;
END //
DELIMITER ;

```

7.4 Bloque 4: Disparadores (Triggers)

1. Auditoría de Pagos:

Listing 34: src/sol_b4_q1.sql

```

DELIMITER //
CREATE TRIGGER audit_pay AFTER UPDATE ON payment FOR EACH ROW
BEGIN
    IF OLD.amount <> NEW.amount THEN
        INSERT INTO audit_payments(payment_id, old_amt, new_amt)
        VALUES (OLD.payment_id, OLD.amount, NEW.amount);
    END IF;
END //
DELIMITER ;

```

2. Protección de Actores:

Listing 35: src/sol_b4_q2.sql

```

DELIMITER //
CREATE TRIGGER protect_actors BEFORE DELETE ON actor FOR EACH ROW
BEGIN
    DECLARE v_count INT;
    SELECT COUNT(*) INTO v_count FROM film_actor WHERE actor_id = OLD.actor_id;
    IF v_count > 20 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Actor demasiado famoso para
        borrar';
    END IF;
END //
DELIMITER ;

```

3. Auto-Mayúsculas:

Listing 36: src/sol_b4_q3.sql

```

DELIMITER //
CREATE TRIGGER cust_upper BEFORE UPDATE ON customer FOR EACH ROW

```

```
BEGIN
    SET NEW.first_name = UPPER(NEW.first_name);
END //
DELIMITER ;
```

4. Control de Stock Mínimo:

Listing 37: src/sol_b4_q4.sql

```
DELIMITER //
CREATE TRIGGER check_min_inventory BEFORE INSERT ON rental FOR EACH ROW
BEGIN
    DECLARE v_count INT;
    SELECT COUNT(*) INTO v_count FROM inventory WHERE inventory_id = NEW.
        inventory_id;
    IF v_count = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Copia de inventario no
            existente';
    END IF;
END //
DELIMITER ;
```

5. Histórico de Email:

Listing 38: src/sol_b4_q5.sql

```
DELIMITER //
CREATE TRIGGER email_hist BEFORE UPDATE ON customer FOR EACH ROW
BEGIN
    IF OLD.email <> NEW.email THEN
        INSERT INTO email_history(customer_id, old_email) VALUES (OLD.customer_id,
            OLD.email);
    END IF;
END //
DELIMITER ;
```

7.5 Bloque 5: Cursores y Handlers

1. Resumen de Ventas Staff:

Listing 39: src/sol_b5_q1.sql

```
DELIMITER //
CREATE PROCEDURE total_staff_sales()
BEGIN
    DECLARE v_id INT;
    DECLARE v_total DECIMAL(10,2);
    DECLARE v_fin INT DEFAULT FALSE;
    DECLARE cur CURSOR FOR SELECT staff_id FROM staff;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_id;
        IF v_fin THEN LEAVE read_loop; END IF;
        SELECT SUM(amount) INTO v_total FROM payment WHERE staff_id = v_id;
```

```

        INSERT INTO staff_report(id, total) VALUES (v_id, v_total);
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

```

2. Normalizador de Nombres:

Listing 40: src/sol_b5_q2.sql

```

DELIMITER //
CREATE PROCEDURE capitalize_actors()
BEGIN
    DECLARE v_id INT;
    DECLARE v_fname VARCHAR(45);
    DECLARE v_fin INT DEFAULT FALSE;
    DECLARE cur CURSOR FOR SELECT actor_id, first_name FROM actor;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;
    OPEN cur;
    loop1: LOOP
        FETCH cur INTO v_id, v_fname;
        IF v_fin THEN LEAVE loop1; END IF;
        UPDATE actor SET first_name = CONCAT(UPPER(LEFT(v_fname,1)), LOWER(
            SUBSTRING(v_fname,2)))
        WHERE actor_id = v_id;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

```

3. Informe Ingresos por Categoría:

Listing 41: src/sol_b5_q3.sql

```

DELIMITER //
CREATE PROCEDURE cat_report()
BEGIN
    DECLARE v_name VARCHAR(25);
    DECLARE v_rev DECIMAL(10,2);
    DECLARE v_fin INT DEFAULT FALSE;
    DECLARE cur CURSOR FOR SELECT name FROM category;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;
    OPEN cur;
    loop_r: LOOP
        FETCH cur INTO v_name;
        IF v_fin THEN LEAVE loop_r; END IF;
        SELECT SUM(amount) INTO v_rev FROM payment JOIN rental USING(rental_id)
        JOIN inventory USING(inventory_id) JOIN film_category USING(film_id)
        JOIN category USING(category_id) WHERE name = v_name;
        INSERT INTO cat_stats VALUES (v_name, v_rev);
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

```

4. Aviso de Devolución:

Listing 42: src/sol_b5_q4.sql

```

DELIMITER //
CREATE PROCEDURE overdue_report()
BEGIN
    DECLARE v_id INT;
    DECLARE v_fin INT DEFAULT FALSE;
    DECLARE cur CURSOR FOR SELECT rental_id FROM rental WHERE return_date IS NULL
        AND rental_date < DATE_SUB(NOW(), INTERVAL 7 DAY);
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;
    OPEN cur;
    loop_o: LOOP
        FETCH cur INTO v_id;
        IF v_fin THEN LEAVE loop_o; END IF;
        INSERT INTO overdue_logs(rental_id) VALUES (v_id);
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

```

5. Reemplazo Masivo de Descripción:

Listing 43: src/sol_b5_q5.sql

```

DELIMITER //
CREATE PROCEDURE replace_desc(IN p_old VARCHAR(50), IN p_new VARCHAR(50))
BEGIN
    DECLARE v_id INT;
    DECLARE v_desc TEXT;
    DECLARE v_fin INT DEFAULT FALSE;
    DECLARE cur CURSOR FOR SELECT film_id, description FROM film;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;
    OPEN cur;
    loop_res: LOOP
        FETCH cur INTO v_id, v_desc;
        IF v_fin THEN LEAVE loop_res; END IF;
        IF v_desc LIKE CONCAT('%', p_old, '%') THEN
            UPDATE film SET description = REPLACE(v_desc, p_old, p_new) WHERE
                film_id = v_id;
        END IF;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

```